

DBMS-ORDERS AND ORDER_DETAILS MANAGEMENT

PROJECT TITLE :SHOPWITHNYKAA

INTRODUCTION

The **Nykaa E-Commerce Order Management System** is a database module designed to manage customer orders and their corresponding product details. The system uses two main tables, **Orders** and **Order_Details**, to store order information and product-level details. The tables are related through **Order_ID**, allowing order records and their product details to be managed together

OBJECTIVE

The main objectives of this project are:

- To create and manage the **Orders** table.
- To create and manage the **Order_Details** table.
- To store customer order and product details systematically.
- To establish a relationship between **Orders** and **Order_Details**.
- To perform order insertion and modification operations.
- To generate **Customer Order History** and **Customer-wise Order Summary** reports.

1. Database Design

1.1 Design the Orders table

Attribute	Constraint
Order_ID	Primary Key
Customer_ID	Foreign Key
Order_Date	NOT NULL
Shipping_Address	NOT NULL
Order_Status	NOT NULL
Total_Amount	—

1.2 Order_Details Table

Attribute	Constraint
Order_Detail_ID	Primary Key
Order_ID	Foreign Key
Product_ID	Foreign Key
Quantity	NOT NULL

Unit_Price	NOT NULL
Discount	—
Total_Price	—

Table Relationships

- Orders.Order_ID → Primary Key
- Order_Details.Order_Detail_ID → Primary Key
- Order_Details.Order_ID → Foreign Key referencing Orders.Order_ID

Relationship: One-to-Many (1:M) — one order can have multiple order detail records

3. Table Creation

3.1 Create Orders Table

```
SQL> CREATE TABLE Orders (
  2     Order_ID NUMBER PRIMARY KEY,
  3     Customer_ID NUMBER,
  4     Order_Date DATE NOT NULL,
  5     Shipping_Address VARCHAR2(200) NOT NULL,
  6     Order_Status VARCHAR2(30) NOT NULL,
  7     Total_Amount NUMBER(10,2),
  8     CONSTRAINT fk_orders_customer
  9         FOREIGN KEY (Customer_ID)
 10         REFERENCES Customer(Customer_ID)
 11 );
```

Table created.

3.2 Create Order Details

```
SQL> CREATE TABLE Order_Details (
  2     Order_Detail_ID NUMBER PRIMARY KEY,
  3     Order_ID NUMBER,
  4     Product_ID NUMBER,
  5     Quantity NUMBER NOT NULL,
  6     Unit_Price NUMBER(10,2) NOT NULL,
  7     Discount NUMBER(10,2),
  8     Total_Price NUMBER(10,2),
  9     CONSTRAINT fk_orderdetails_order
 10         FOREIGN KEY (Order_ID)
 11         REFERENCES Orders(Order_ID),
 12     CONSTRAINT fk_orderdetails_product
 13         FOREIGN KEY (Product_ID)
 14         REFERENCES Product(Product_ID)
 15 );
```

Table created.

4. Order Data Insertion

```
SQL> INSERT INTO Orders VALUES
  2 (1, 101, DATE '2026-09-01', 'Chennai, Tamil Nadu', 'Delivered', 1500.00);
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2 (2, 102, DATE '2026-09-03', 'Tambaram, Chennai', 'Shipped', 2300.00);
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2 (3, 103, DATE '2026-09-05', 'Chrompet, Chennai', 'Processing', 1750.00);
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2 (4, 104, DATE '2026-09-08', 'Velachery, Chennai', 'Delivered', 3200.00);
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES
  2 (5, 105, DATE '2026-09-10', 'Guindy, Chennai', 'Pending', 950.00);
1 row created.
```

5. Order Details Data Insertion

```
SQL> INSERT INTO Order_Details VALUES
  2 (601, 1, 1001, 2, 599.00, 0.00, 1198.00);
1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
  2 (602, 2, 1002, 2, 499.00, 0.00, 998.00);
1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
  2 (603, 3, 1003, 1, 399.00, 0.00, 399.00);
1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
  2 (604, 4, 1004, 2, 1299.00, 0.00, 2598.00);
1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES
  2 (605, 5, 1005, 1, 349.00, 0.00, 349.00);
1 row created.
```

6. Order Modification Operations

6.1 Modify Order Amount

```
SQL> UPDATE Orders
  2 SET Total_Amount = 1600.00
  3 WHERE Order_ID = 1;
1 row updated.

SQL> 1 row updated.
SQL> SELECT * FROM Orders WHERE Order_id = 1;

  ORDER_ID CUSTOMER_ID ORDER_DAT
-----
SHIPPING_ADDRESS
-----
ORDER_STATUS                                TOTAL_AMOUNT
-----
          1          101 01-SEP-26
Chennai, Tamil Nadu
Delivered                                1600
```

6.2 Modify Order Date

```
SQL> UPDATE Orders
  2 SET Order_Date = DATE '2026-09-02'
  3 WHERE Order_ID = 1;

1 row updated.

SQL> SELECT * FROM Orders WHERE Order_ID = 1;

  ORDER_ID CUSTOMER_ID ORDER_DAT
-----
SHIPPING_ADDRESS
-----
ORDER_STATUS              TOTAL_AMOUNT
-----
          1          101 02-SEP-26
Chennai, Tamil Nadu
Delivered                      1600
```

7. Order Details Modification

```
SQL> UPDATE Order_Details
  2 SET Quantity = 3,
  3     Total_Price = 1797.00
  4 WHERE Order_Detail_ID = 601;

1 row updated.

SQL> SELECT * FROM Order_Details WHERE Order_Detail_ID = 601;

ORDER_DETAIL_ID  ORDER_ID PRODUCT_ID  QUANTITY UNIT_PRICE  DISCOUNT
-----
TOTAL_PRICE
-----
          601          1      1001          3        599          0
          1797
```

8. Customer Order History Report

```
SQL> SELECT
  2     o.Customer_ID,
  3     o.Order_ID,
  4     o.Order_Date,
  5     o.Order_Status,
  6     od.Product_ID,
  7     od.Quantity,
  8     od.Unit_Price,
  9     od.Total_Price
10 FROM Orders o
11 JOIN Order_Details od
12 ON o.Order_ID = od.Order_ID
13 ORDER BY o.Customer_ID, o.Order_Date;

CUSTOMER_ID  ORDER_ID ORDER_DAT ORDER_STATUS
-----
  QUANTITY UNIT_PRICE TOTAL_PRICE
-----
          101          1 02-SEP-26 Delivered          1001
            3          599          1797
          102          2 03-SEP-26 Shipped          1002
            2          499          998
          103          3 05-SEP-26 Processing          1003
            1          399          399

CUSTOMER_ID  ORDER_ID ORDER_DAT ORDER_STATUS
-----
  QUANTITY UNIT_PRICE TOTAL_PRICE
-----
          104          4 08-SEP-26 Delivered          1004
            2          1299          2598
          105          5 10-SEP-26 Pending          1005
            1          349          349
```

9. Customer-wise Order Summary

```
SQL> SELECT
2     Customer_ID,
3     COUNT(Order_ID) AS Total_Orders,
4     SUM(Total_Amount) AS Total_Amount
5 FROM Orders
6 GROUP BY Customer_ID
7 ORDER BY Customer_ID;
```

CUSTOMER_ID	TOTAL_ORDERS	TOTAL_AMOUNT
101	1	1600
102	1	2300
103	1	1750
104	1	3200
105	1	950

Conclusion

The **Orders** and **Order_Details** tables were successfully designed and implemented for the Nykaa e-commerce database. The project demonstrated how customer orders and their product details can be stored and managed using SQL. Order insertion, modification, and reporting operations were performed successfully. The **Customer Order History** and **Customer-wise Order Summary** reports provide a clear view of customer purchases and order amounts, completing the Order Management module.